

Sistema Inteligente de Detecção de Sobrecarga — Versão Completa

Engenharia Elétrica · Algoritmos e Programação · Matheus Leal Peres

Simula corrente AC de 60Hz com injeção de sobrecarga sustentada e transientes de corrente. Detecta e classifica cada evento, gera o gráfico de monitoramento e exporta um relatório CSV com os marcadores de falha (início, fim, pico e classificação).

Código-fonte (Python)

```
# =====
# Sistema Inteligente de Detecção de Sobrecarga - Versão Completa
# Engenharia Elétrica - Monitoramento de Corrente em Tempo Real
# =====
#
# Módulos (conforme a estrutura da apresentação):
# 1. Sinal:      simulação de corrente AC senoidal (NumPy)
# 2. Falha:      injeção controlada de sobrecarga + transientes
# 3. Detector:    comparação vetorial contra o limite de 15A
# 4. Saída:      gráfico (PNG) + relatório de falhas (CSV)

import numpy as np
import matplotlib.pyplot as plt
import csv

# --- Parâmetros do sistema (conforme os slides) ---
LIMITE_SEGURANCA_A = 15.0      # Limite de segurança crítico
FREQUENCIA_HZ = 60             # Frequência da rede (AC)
AMPLITUDE_NORMAL_A = 10.0      # Amplitude da corrente em operação normal
DURACAO_S = 2.0                # Duração total da simulação
TAXA_AMOSTRAGEM = 4000         # Amostras por segundo

# -----
# Módulo 1: Sinal
# -----
def gerar_sinal_corrente(duracao_s, taxa_amostragem, amplitude_a, frequencia_hz):
    """Gera o dataset temporal: corrente AC senoidal em operação normal."""
    t = np.linspace(0, duracao_s, int(duracao_s * taxa_amostragem), endpoint=False)
    corrente = amplitude_a * np.sin(2 * np.pi * frequencia_hz * t)
    return t, corrente

# -----
# Módulo 2: Falha (sobrecarga + transientes)
# -----
def injetar_sobrecarga_sustentada(t, corrente, inicio_s, fim_s, amplitude_extra_a):
    """Sobrecarga persistente: eleva a amplitude num intervalo de tempo
    contínuo, simulando uma demanda excessiva sustentada (ex.: muitos
    equipamentos ligados ao mesmo tempo)."""
    corrente_com_falha = corrente.copy()
    mascara = (t >= inicio_s) & (t <= fim_s)
    corrente_com_falha[mascara] += amplitude_extra_a
    return corrente_com_falha

def injetar_transiente(t, corrente, instante_s, amplitude_pico_a, duracao_pico_s=0.005):
    """Transiente: pico breve e abrupto de corrente, simulando uma
    chave de partida de motor ou descarga atmosférica próxima."""
    corrente_com_falha = corrente.copy()
    mascara = (t >= instante_s) & (t <= instante_s + duracao_pico_s)
    corrente_com_falha[mascara] += amplitude_pico_a
    return corrente_com_falha
# -----
```

```

# Módulo 3: Detector
# -----
def detectar_sobrecarga(t, corrente, limite_a, intervalo_agrupamento_s=0.03):
    """Compara vetorialmente a amplitude do sinal contra o limite.

    Como a corrente é um sinal AC, ela cruza o limite e volta a cada
    ciclo durante uma sobrecarga sustentada – por isso, pontos de
    alarme separados por menos de `intervalo_agrupamento_s` são
    tratados como o MESMO evento (e não como vários transientes).

    Retorna a máscara booleana de alarme e a lista de eventos
    (início, fim, pico) já agrupados.
    """
    alarme = np.abs(corrente) > limite_a
    indices_alarme = np.where(alarme)[0]

    eventos = []
    if len(indices_alarme) == 0:
        return alarme, eventos

    inicio_idx = indices_alarme[0]
    fim_idx = indices_alarme[0]

    for idx in indices_alarme[1:]:
        gap_s = t[idx] - t[fim_idx]
        if gap_s <= intervalo_agrupamento_s:
            # Mesmo evento: só estende o fim
            fim_idx = idx
        else:
            # Gap grande: fecha o evento anterior e abre um novo
            pico = np.max(np.abs(corrente[inicio_idx:fim_idx + 1]))
            eventos.append({
                "inicio_s": round(t[inicio_idx], 4),
                "fim_s": round(t[fim_idx], 4),
                "pico_a": round(pico, 2),
            })
            inicio_idx = idx
            fim_idx = idx

    # Fecha o último evento
    pico = np.max(np.abs(corrente[inicio_idx:fim_idx + 1]))
    eventos.append({
        "inicio_s": round(t[inicio_idx], 4),
        "fim_s": round(t[fim_idx], 4),
        "pico_a": round(pico, 2),
    })

    return alarme, eventos

# -----
# Módulo 4: Saída (gráfico + relatório)
# -----
def gerar_grafico(t, corrente, alarme, limite_a, caminho_saida):
    plt.figure(figsize=(12, 5))
    plt.plot(t, corrente, label="Corrente (A)", color="#3b82f6", linewidth=1.0)
    plt.axhline(y=limite_a, color="red", linestyle="--", linewidth=1, label=f"Limite ({limite_a} A)")
    plt.axhline(y=-limite_a, color="red", linestyle="--", linewidth=1, label=f"Limite (-{limite_a} A)")
    plt.scatter(t[alarme], corrente[alarme], color="orange", s=8, zorder=5, label="Sobrecarga / transiente")

    plt.title("Sistema Inteligente de Detecção de Sobrecarga – Monitoramento em Tempo Real")
    plt.xlabel("Tempo (s)")
    plt.ylabel("Corrente (A)")
    plt.legend(loc="upper right")
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.savefig(caminho_saida, dpi=150)
    plt.close()

def gerar_relatorio_csv(eventos, caminho_saida):

```

```

"""Exporta os marcadores de falha em formato CSV (Módulo Saída)."""
with open(caminho_saida, "w", newline="", encoding="utf-8") as f:
    escritor = csv.writer(f)
    escritor.writerow(["evento", "inicio_s", "fim_s", "duracao_s", "pico_corrente_a", "classificacao"])
    for i, ev in enumerate(eventos, start=1):
        duracao = round(ev["fim_s"] - ev["inicio_s"], 4)
        # Heurística simples de classificação: eventos muito curtos (< 20ms)
        # são tratados como transientes; os demais, como sobrecarga sustentada.
        classificacao = "transiente" if duracao < 0.02 else "sobrecarga_sustentada"
        escritor.writerow([i, ev["inicio_s"], ev["fim_s"], duracao, ev["pico_a"], classificacao])

# -----
# Execução principal
# -----
def main():
    # 1. Gera o sinal normal de corrente
    t, corrente = gerar_sinal_corrente(
        DURACAO_S, TAXA_AMOSTRAGEM, AMPLITUDE_NORMAL_A, FREQUENCIA_HZ
    )

    # 2. Injeta as falhas: uma sobrecarga sustentada e dois transientes
    corrente = injetar_sobrecarga_sustentada(t, corrente, inicio_s=0.5, fim_s=0.65, amplitude_extra_a=7.0)
    corrente = injetar_transiente(t, corrente, instante_s=1.0, amplitude_pico_a=12.0)
    corrente = injetar_transiente(t, corrente, instante_s=1.6, amplitude_pico_a=9.0)

    # 3. Detecta os eventos de sobrecarga
    alarme, eventos = detectar_sobrecarga(t, corrente, LIMITE_SEGURANCA_A)

    # 4. Gera as saídas (gráfico + CSV)
    gerar_grafico(t, corrente, alarme, LIMITE_SEGURANCA_A, "grafico_deteccao_completo.png")
    gerar_relatorio_csv(eventos, "relatorio_falhas.csv")

    # --- Relatório no console ---
    print("=" * 60)
    print("SISTEMA INTELIGENTE DE DETECÇÃO DE SOBRECARGA")
    print("=" * 60)
    print(f"Limite de segurança: {LIMITE_SEGURANCA_A} A")
    print(f"Frequência da rede: {FREQUENCIA_HZ} Hz")
    print(f"Duração simulada: {DURACAO_S} s ({len(t)} amostras)")
    print(f"Pico máximo registrado: {np.max(np.abs(corrente)):.2f} A")
    print(f"\nTotal de eventos de sobrecarga detectados: {len(eventos)}")
    print("-" * 60)
    for i, ev in enumerate(eventos, start=1):
        duracao = round(ev["fim_s"] - ev["inicio_s"], 4)
        tipo = "transiente" if duracao < 0.02 else "sobrecarga sustentada"
        print(f"Evento {i}: {ev['inicio_s']}s - {ev['fim_s']}s "
              f"| pico {ev['pico_a']} A | {tipo}")
    print("-" * 60)
    print("\nArquivos gerados:")
    print(" - grafico_deteccao_completo.png")
    print(" - relatorio_falhas.csv")

if __name__ == "__main__":
    main()

```

Saída no console

```
=====
SISTEMA INTELIGENTE DE DETECÇÃO DE SOBRECARGA
=====
Limite de segurança: 15.0 A
Frequência da rede: 60 Hz
Duração simulada: 2.0 s (8000 amostras)
Pico máximo registrado: 22.00 A

Total de eventos de sobrecarga detectados: 3
-----
Evento 1: 0.5025s - 0.639s | pico 17.0 A | sobrecarga sustentada
Evento 2: 1.001s - 1.0048s | pico 22.0 A | transiente
Evento 3: 1.6018s - 1.605s | pico 19.0 A | transiente
-----

Arquivos gerados:
- grafico_deteccao_completo.png
- relatorio_falhas.csv
```

Gráfico gerado

